

ECE 468: Lab 1 Report

UNIVERSITY OF
Waterloo



Department of Electrical and Computer Engineering

A Report Prepared For:

The University of Waterloo
Dr. Gennaro Notomista
ECE468

Prepared By:

Hanhua Ge
Ameen Aydan
150 University Ave W.
Waterloo, Ontario, N2N 2N2

May 26th, 2026

Table of Contents

The Workspace	3
Workspace Design	3
Workspace Validation	3
Trajectories	4
Trajectory Derivation	4
Error Analysis	5
Appendix.....	7
References	10

The Workspace

Workspace Design

The function “is_point_in_workspace” takes any point and returns a bool to show whether it is sitting in the workspace defined in the lab. The path that the end effector takes to get to that point has the possibility of exiting the constrained workspace. To prevent this from happening, another function is created called “is_path_in_workspace”. This uses the numpy library to create a straight-line vector and divide it into 100 points. Each of these points lies along the path that the robot follows and is checked with “is_point_in_workspace” to ensure the movement remains within the workspace. This does not apply to the path between homing and the subsequent point lying in the pathway.

Since any point given to the robot will generate a path, it’s only necessary to use the latter function. If the robot path of travel is not within the workspace, the function returns a false and exits before the path can execute.

Workspace Validation

To validate that the workspace function works, a series of points are fed to the robot in the simulator. The robot moves from one point to the next to demonstrate the validity of the trajectory or point being within the workspace. If the point or path fails to be within the workspace, even partially, the move is not executed and a statement is returned as, “Rejected target {target_xyz}: outside workspace”.

Table 1 below shows, in sequence, the points that are given to the robot to validate that it will remain within the workspace. It also shows the expected behaviour depending on whether it’s in the workspace or not. All of these points were tested and validated with the physical robot.

Table 1: Points tested to see if it lies within the workspace, and whether the function works to filter it out.

Position #	Position (X, Y, Z)	Point in workspace?	Path in workspace?
1	200, 0, -50	Yes	Not checked
2	(180, 150, -80)	Yes	Yes
3	(100, 0, -50)	No, radius too small	Not checked
4	(270, 0, -50)	No, radius too large	Not checked
5	(200, 0, -130)	No, height too small	Not checked

Trajectories

Trajectory Derivation

The trajectory used for Part 3 is a circular Cartesian trajectory in the horizontal x-y plane at a constant height. The circle is defined by a center point, radius, and angular parameter:

$$\mathbf{c} = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} 200 \\ 0 \\ -10 \end{bmatrix} \text{ mm}, \quad R = 50 \text{ mm}$$

$$x(t) = x_c + R \cos(t)$$

$$y(t) = y_c + R \sin(t)$$

$$z(t) = z_c$$

We can show that this trajectory is twice differentiable, because its second derivative is smooth.

$$\frac{d^2x}{dt^2}(t) = -R \cos(t)$$

$$\frac{d^2y}{dt^2}(t) = -R \sin(t)$$

$$\frac{d^2z}{dt^2}(t) = 0$$

Since all second derivatives exist and are continuous, the trajectory is twice differentiable.

For N trajectory points, the angle for point i is chosen as:

$$t = \theta_i = \frac{2\pi i}{N}, \quad i = 0, 1, 2, \dots, N-1$$

The corresponding Cartesian target point is:

$$\mathbf{p}_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} = \begin{bmatrix} x_c + R \cos \theta_i \\ y_c + R \sin \theta_i \\ z_c \end{bmatrix}$$

Substituting the selected values gives:

$$x_i = 200 + 50 \cos\left(\frac{2\pi i}{N}\right), \quad y_i = 50 \sin\left(\frac{2\pi i}{N}\right), \quad z_i = -10$$

This produces equally spaced target points around a circle of radius 50 mm. The trajectory remains within the restricted workspace because:

$$150 \leq x_i \leq 250, \quad -50 \leq y_i \leq 50, \quad z_i = -10$$

and the radial distance from the base satisfies:

$$150 \leq \sqrt{x_i^2 + y_i^2} \leq 250$$

Therefore, every target point satisfies the workspace limits:

$$-120 \leq z_i \leq 0, \quad 140 \leq \sqrt{x_i^2 + y_i^2} \leq 260, \quad x_i \geq 0$$

In the implementation, these points are generated using N = 50 equally spaced values of theta_i, and each target is passed through safe_move_to_xyz() before execution. This means the trajectory

is only executed when both the target point and the straight-line path to that point remain inside the valid workspace. Additionally, the trajectory is twice differentiable with respect to time.

Error Analysis

As instructed by the lab instructor Mr. Cooper, we ran a total of 8 sets of data instead of 10 as mentioned in manual, because of the lack of manipulator availability.

It was found that the error of real manipulator is very tiny. Furthermore, comparing the error data between these 8 separate experiments showed that they were the same, which is why we omit the 8 different runs from this report. The lab instructor explained this finding was because the robotic arm is a highly precise device. This is also likely due to the pose values being obtained from the controller's internal kinematic model rather than from an external measurement system, such as camera vision. In fact, there isn't an external measurement system which can measure the real position from outside (such as encoders on each joint of the arm).

The python script outputs the target position and real position of each sampling point in real time and calculates the Euclidean distance between them as an error for that point. Table 3 in the appendix is a list of 50 points' target position, real position, and errors. Figure 1 and 2 are graphs that show the trajectory and error of each trajectory point respectively.

$$p_{it} = (x_{it}, y_{it}, z_{it})^T$$

$$p_{ir} = (x_{ir}, y_{ir}, z_{ir})^T$$

$$e_i = \|p_{it} - p_{ir}\| = \sqrt{(x_{it} - x_{ir})^2 + (y_{it} - y_{ir})^2 + (z_{it} - z_{ir})^2}$$

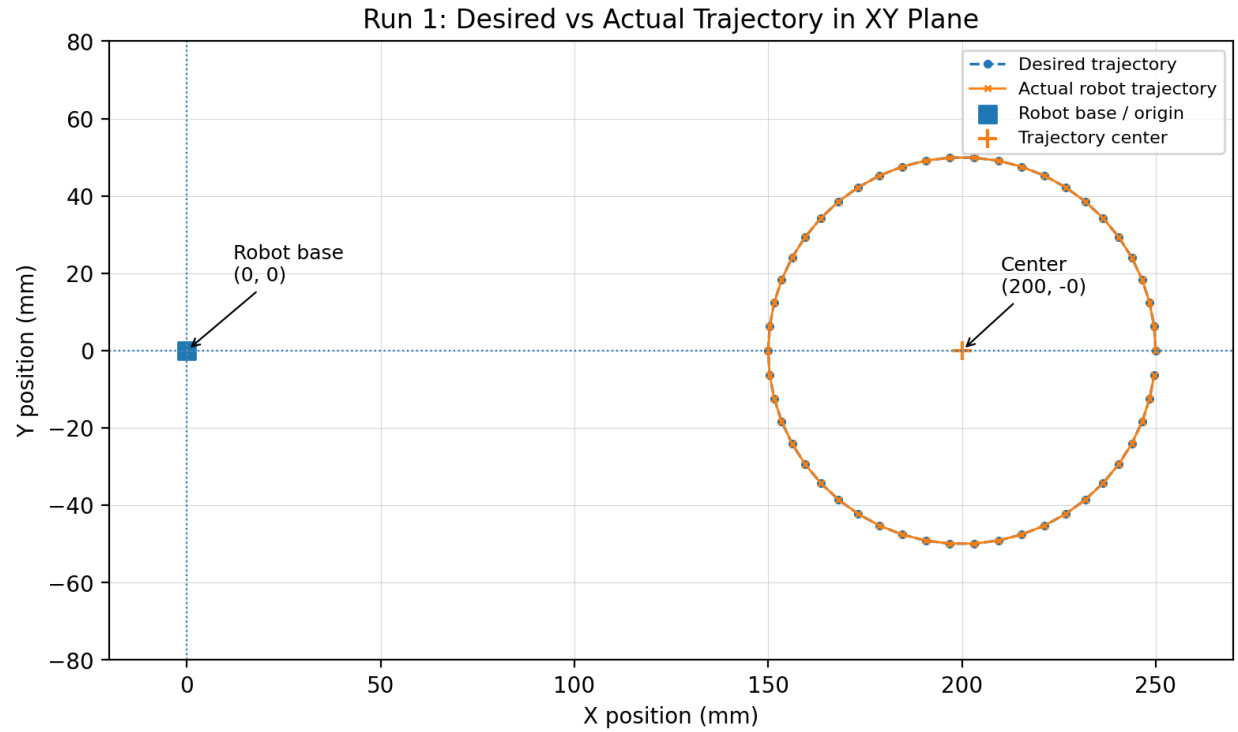


Figure 1: Graph showing the actual and desired trajectory of the end effector in the XY plane

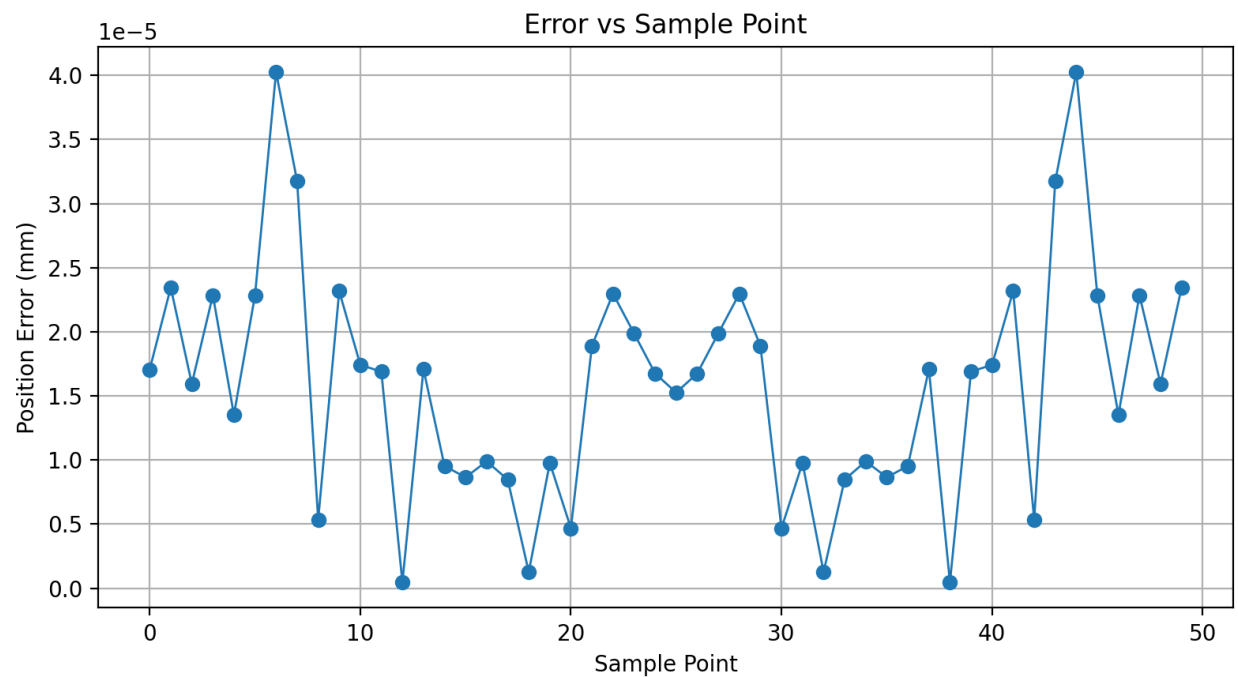


Figure 2: Graph showing the error at each point along the trajectory

According to calculation, the mean error of 50 points is 0.0000159 mm, with maximum up to 0.0000403 mm, and minimum down to 0.000000463 mm. This is shown in table 2 below.

Table 2: Mean, max and min error from actual robot position

Mean error	0.0000159
Maximum error	0.0000403
Minimum error	0.000000463

Nevertheless, all 8 runs produced exactly same reported positions for all 50 sample points. Therefore, the measured run-to-run variation is 0 under this condition. This result likely reflects the limitation of using the robot controller's internal pose output rather than some external measurement system.

The Dobot magician specification (Robot lab) mentions that the robot has a position repeatability of 0.2mm. The results of this report demonstrate that this is far more than what was measured, as is likely due to the lack of an external measurement component.

Appendix

Table 3: Table of the target position, the actual position and the measured error between real and target for the end effector during the trajectory runs.

Point	Target Position	Real Position	Error (mm)
0	(250.0000, 0.0000, -10.0000)	(249.9999847412, 0.0000000000, -9.9999923706)	0.0000170598
1	(249.6057, 6.2667, -10.0000)	(249.6057128906, 6.2666616440, -10.0000076294)	0.0000234509
2	(248.4292, 12.4345, -10.0000)	(248.4291534424, 12.4344940186, -9.9999847412)	0.0000159448
3	(246.4888, 18.4062, -10.0000)	(246.4888458252, 18.4062271118, -9.9999923706)	0.0000228485
4	(243.8153, 24.0877, -10.0000)	(243.8153228760, 24.0876827240, -10.0000076294)	0.0000135264
5	(240.4508, 29.3893, -10.0000)	(240.4508666992, 29.3892631531, -10.0000152588)	0.0000228354
6	(236.4484, 34.2274, -10.0000)	(236.4483947754, 34.2273483276, -9.9999847412)	0.0000402572
7	(231.8712, 38.5257, -10.0000)	(231.8712310791, 38.5256652832, -10.0000000000)	0.0000317478
8	(226.7913, 42.2164, -10.0000)	(226.7913360596, 42.2164001465, -10.0000000000)	0.0000053478
9	(221.2890, 45.2414, -10.0000)	(221.2889862061, 45.2413558960, -10.0000076294)	0.0000231664
10	(215.4508, 47.5528, -10.0000)	(215.4508666992, 47.5528297424, -10.0000000000)	0.0000174288

11	(209.3691, 49.1144, -10.0000)	(209.3690795898, 49.1143684387, -9.9999923706)	0.0000168867
12	(203.1395, 49.9013, -10.0000)	(203.1395263672, 49.9013366699, -10.0000000000)	0.0000004631
13	(196.8605, 49.9013, -10.0000)	(196.8604888916, 49.9013328552, -10.0000076294)	0.0000170876
14	(190.6309, 49.1144, -10.0000)	(190.6309356689, 49.1143569946, -9.9999923706)	0.0000095328
15	(184.5492, 47.5528, -10.0000)	(184.5491485596, 47.5528221130, -10.0000076294)	0.0000086530
16	(178.7110, 45.2414, -10.0000)	(178.7110443115, 45.2413482666, -10.0000000000)	0.0000098999
17	(173.2087, 42.2164, -10.0000)	(173.2086639404, 42.2163963318, -9.9999923706)	0.0000084748
18	(168.1288, 38.5257, -10.0000)	(168.1287994385, 38.5256614685, -10.0000000000)	0.0000012661
19	(163.5516, 34.2274, -10.0000)	(163.5515747070, 34.2273559570, -9.9999923706)	0.0000097769
20	(159.5492, 29.3893, -10.0000)	(159.5491485596, 29.3892669678, -10.0000000000)	0.0000046813
21	(156.1847, 24.0877, -10.0000)	(156.1846771240, 24.0876827240, -9.9999847412)	0.0000189099
22	(153.5112, 18.4062, -10.0000)	(153.5111541748, 18.4062252045, -9.9999923706)	0.0000229714
23	(151.5708, 12.4345, -10.0000)	(151.5708618164, 12.4344940186, -10.0000000000)	0.0000198757
24	(150.3943, 6.2667, -10.0000)	(150.3942718506, 6.2666616440, -9.9999847412)	0.0000167531
25	(150.0000, 0.0000, -10.0000)	(150.0000000000, 0.0000000000, -9.9999847412)	0.0000152588
26	(150.3943, -6.2667, -10.0000)	(150.3942718506, -6.2666616440, -9.9999847412)	0.0000167531
27	(151.5708, -12.4345, -10.0000)	(151.5708618164, - 12.4344940186, -10.0000000000)	0.0000198757
28	(153.5112, -18.4062, -10.0000)	(153.5111541748, - 18.4062252045, -9.9999923706)	0.0000229714
29	(156.1847, -24.0877, -10.0000)	(156.1846771240, - 24.0876827240, -9.9999847412)	0.0000189099
30	(159.5492, -29.3893, -10.0000)	(159.5491485596, - 29.3892669678, -10.0000000000)	0.0000046813
31	(163.5516, -34.2274, -10.0000)	(163.5515747070, - 34.2273559570, -9.9999923706)	0.0000097769

32	(168.1288, -38.5257, -10.0000)	(168.1287994385, - 38.5256614685, -10.0000000000)	0.0000012661
33	(173.2087, -42.2164, -10.0000)	(173.2086639404, - 42.2163963318, -9.9999923706)	0.0000084748
34	(178.7110, -45.2414, -10.0000)	(178.7110443115, - 45.2413482666, -10.0000000000)	0.0000098999
35	(184.5492, -47.5528, -10.0000)	(184.5491485596, - 47.5528221130, -10.0000076294)	0.0000086530
36	(190.6309, -49.1144, -10.0000)	(190.6309356689, - 49.1143569946, -9.9999923706)	0.0000095328
37	(196.8605, -49.9013, -10.0000)	(196.8604888916, - 49.9013328552, -10.0000076294)	0.0000170876
38	(203.1395, -49.9013, -10.0000)	(203.1395263672, - 49.9013366699, -10.0000000000)	0.0000004631
39	(209.3691, -49.1144, -10.0000)	(209.3690795898, - 49.1143684387, -9.9999923706)	0.0000168867
40	(215.4508, -47.5528, -10.0000)	(215.4508666992, - 47.5528297424, -10.0000000000)	0.0000174288
41	(221.2890, -45.2414, -10.0000)	(221.2889862061, - 45.2413558960, -10.0000076294)	0.0000231664
42	(226.7913, -42.2164, -10.0000)	(226.7913360596, - 42.2164001465, -10.0000000000)	0.0000053478
43	(231.8712, -38.5257, -10.0000)	(231.8712310791, - 38.5256652832, -10.0000000000)	0.0000317478
44	(236.4484, -34.2274, -10.0000)	(236.4483947754, - 34.2273483276, -9.9999847412)	0.0000402572
45	(240.4508, -29.3893, -10.0000)	(240.4508666992, - 29.3892631531, -10.0000152588)	0.0000228354
46	(243.8153, -24.0877, -10.0000)	(243.8153228760, - 24.0876827240, -10.0000076294)	0.0000135264
47	(246.4888, -18.4062, -10.0000)	(246.4888458252, - 18.4062271118, -9.9999923706)	0.0000228485
48	(248.4292, -12.4345, -10.0000)	(248.4291534424, - 12.4344940186, -9.9999847412)	0.0000159448
49	(249.6057, -6.2667, -10.0000)	(249.6057128906, -6.2666616440, -10.0000076294)	0.0000234509

References

Robot lab. (n.d.). *Dobot Magician Specifications*. Retrieved May 29, 2026, from <https://www.robotlab.com/hubfs/Dobot/Dobot%20Magician%20Specifications.pdf?t=1515807244104&srsltid=AfmBOorTKzOictjtAzWBUyC9hZW8e9dqPfY74ftk36UAjKDhREYcKt>